

Тема:

Сортування за лінійний час

- Нижня оцінка алгоритмів сортування, заснованих на порівнянні елементів
- Сортування підрахунком
- Сортування за розрядами

Нижня оцінка алгоритмів сортування

Алгоритми сортування, які були розглянуті у попередніх темах, мають одну спільну властивість: при сортуванні використовується тільки **порівняння вхідних елементів**.

Такі алгоритми сортування можна назвати *сортуванням на основі порівнянь* (або просто – *сортування порівняннями*).

Приклади:

Алгоритм сортування	Часова складність
Бульбашкою	$\Theta(n^2)$
Вибором	$\Theta(n^2)$
Вставкою	$\Theta(n^2)$
Злиттям	$\Theta(n \log n)$
Швидке	$\Theta(n \log n)$

Чи можна швидше?

Нижня оцінка алгоритмів сортування

Задача сортування:

Вхід: послідовність n чисел $[a_1, a_2, \dots, a_n]$.

Вихід: перестановка $[a'_1, a'_2, \dots, a'_n]$ вхідної послідовності таким чином, що для всіх її членів виконується співвідношення $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Для елементів вхідної послідовності можливі 5 попарних порівнянь:

$$a_i < a_j;$$

$$a_i > a_j;$$

$$a_i = a_j;$$

$$a_i \leq a_j;$$

$$a_i \geq a_j;$$

Якщо припустити, що всі елементи a_i різні, то залишиться:

$$a_i < a_j;$$

$$a_i > a_j;$$

або

$$a_i < a_j$$

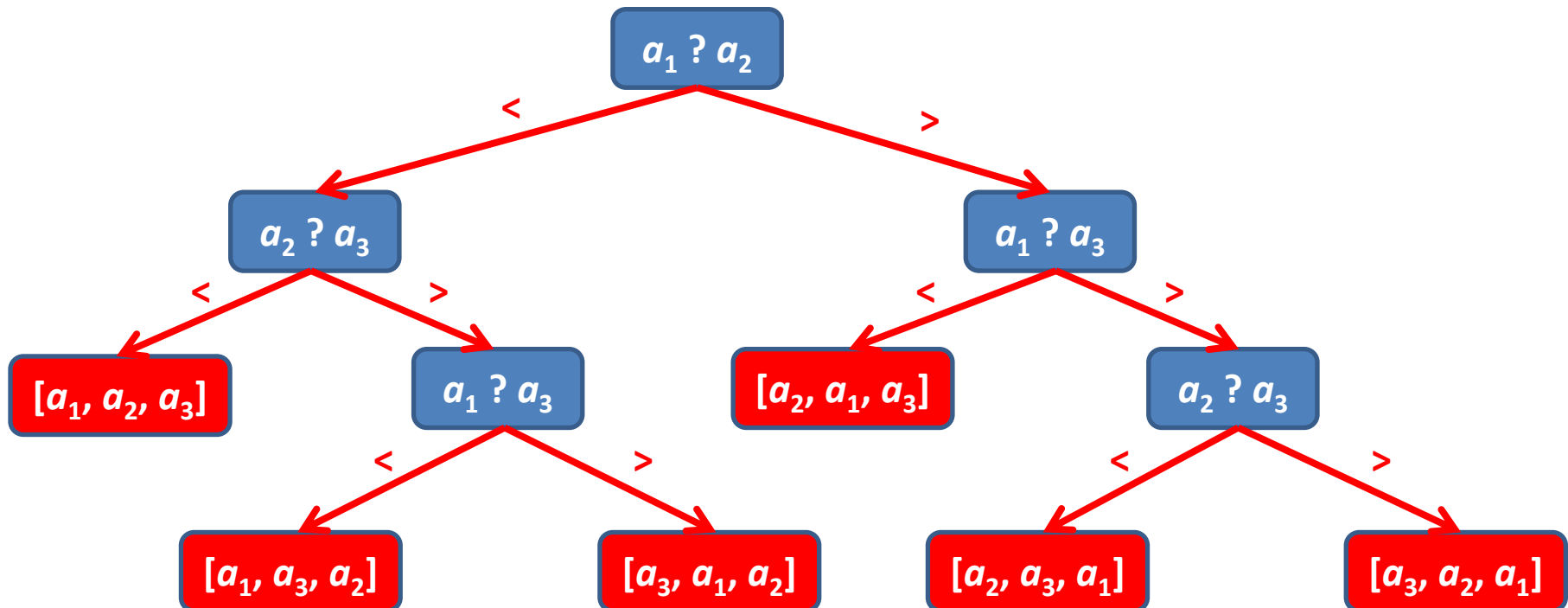
Нижня оцінка алгоритмів сортування

Вивчення алгоритмів сортування порівняннями можна проводити за допомогою *дерев рішень*.

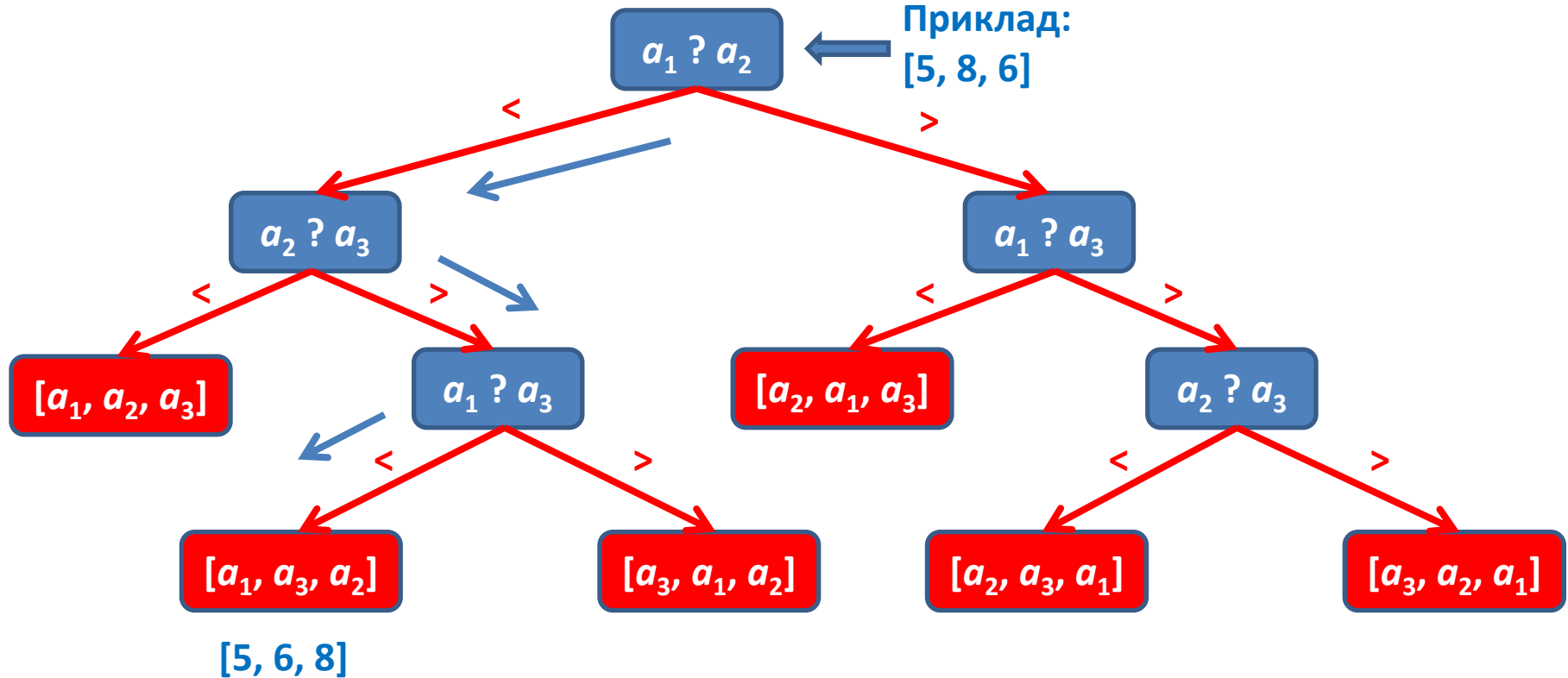
Дерево рішень – це повне бінарне дерево, в якому представлені операції порівняння елементів, які виконуються тим або іншим алгоритмом сортування.

Приклад:

Побудуємо дерево рішень для сортування вектору з трьох елементів: $[a_1, a_2, a_3]$



Нижня оцінка алгоритмів сортування



Отримане дерево рішень має $6 = 3!$ листків, які відповідають усім можливим перестановкам послідовності $[a_1, a_2, a_3]$ (Зауваження: елементи не повторюються).

Зауваження:

В загальному випадку у дереві рішень можуть бути листки з однаковими комбінаціями, тобто кількість листків $l \geq n!$

Нижня оцінка алгоритмів сортування

Теорема.

В найгіршому випадку під час виконання будь-якого алгоритму сортування порівняннями виконується $\Omega(n \lg n)$ порівнянь.

Доведення.

З наведених вище міркувань стає зрозуміло, що для доведення теореми достатньо визначити висоту дерева, в якому кожна перестановка представлена листком. Розглянемо дерево рішень висотою h з l листками, яке відповідає сортуванню n елементів. Оскільки кожна з $n!$ перестановок вхідних елементів зіставляється з одним із листків, $n! \leq l$. Бінарне дерево висотою h має не більше 2^h листків, а тому отримуємо:

$$n! \leq l \leq 2^h.$$

Нижня оцінка алгоритмів сортування

Після логарифмування попереднього виразу, отримуємо:

$$h \geq \log_2 n!. \quad (1)$$

Скористаємось формулою Стірлінга:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \text{ або } n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right).$$

Вираз (1) можна перетворити наступним чином:

$$\begin{aligned} h \geq \log_2 n! &= \log_2 \left(\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right) \right) = \log_2 \sqrt{2\pi n} + \log_2 \left(\frac{n}{e}\right)^n + \log_2 \left(1 + \Theta\left(\frac{1}{n}\right)\right) = \\ &= \frac{1}{2} \log_2 2\pi n + n \log_2 \frac{n}{e} + \log_2 \left(1 + \Theta\left(\frac{1}{n}\right)\right) = \frac{1}{2} \log_2 2\pi n + \underline{n \log_2 n} - n \log_2 e + \log_2 \left(1 + \Theta\left(\frac{1}{n}\right)\right) \\ &= \Theta(n \lg n) \end{aligned}$$

Звідки отримуємо, що $h = \Omega(n \lg n)$. ■

Тобто, алгоритми сортування, засновані на порівнянні елементів в найгіршому випадку працюють за час $n \log n$

Сортування підрахунком

Чи можна швидше?

Розглянемо алгоритм сортування, часова складність якого $T(n) = \Theta(n)$ (лінійна), і де не використовується порівняння елементів.

Основна ідея сортування підрахунком (англ. Counting Sort):

Якщо є k елементів менших за елемент a_i , то елемент a_i повинен знаходитися в $k + 1$ позиції (вважаємо, що всі елементи різні).

Постановка задачі:

Вхід: послідовність n чисел $[a_1, a_2, \dots, a_n]$, де:

- $a_i \in N_0$ – елементи послідовності невід'ємні цілі числа.
- $0 \leq a_i \leq k$ – значення елементів обмежене зверху числом $k = \max[a_1, a_2, \dots, a_n]$.

Вихід: перестановка $[a'_1, a'_2, \dots, a'_n]$ вхідної послідовності таким чином, що для всіх її членів виконується співвідношення $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Обмеження:
за швидкість треба
платити



Сортування підрахунком

$A =$

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $\longrightarrow n = 8, k = 5$

Потрібно створити додаткові масиви (за швидкість треба платити :)

$B =$

1	2	3	4	5	6	7	8

 $\longrightarrow$ Додатковий масив, куди будуть записуватись посортовані елементи

$C =$

0	1	2	3	4	5
0	0	0	0	0	0

 $\longrightarrow$ Додатковий масив, куди буде записуватись кількість елементів певного значення



крок 1: $C[A[j]] = C[A[j]] + 1; 1 \leq j \leq n$

$i =$

0	1	2	3	4	5
2	0	2	3	0	1

 $\longrightarrow$ Кількість елементів $= i$:
2 елементи $= 0$, 0 елементів $= 1$, 2 елементи $= 2$,
3 елементи $= 3$, 0 елементів $= 4$, 1 елемент $= 5$.



крок 2: $C[i] = C[i] + C[i - 1]; 1 \leq i \leq k$

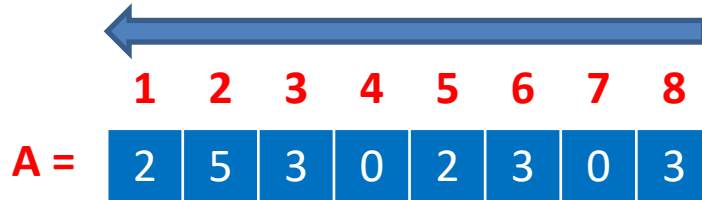
$i =$

0	1	2	3	4	5
2	2	4	7	7	8

 $\longrightarrow$ Кількість елементів $\leq i$:
2 елементи ≤ 0 , 2 елементів ≤ 1 , 4 елементи ≤ 2 ,
7 елементи ≤ 4 , 7 елементів ≤ 4 , 8 елементів ≤ 5 .

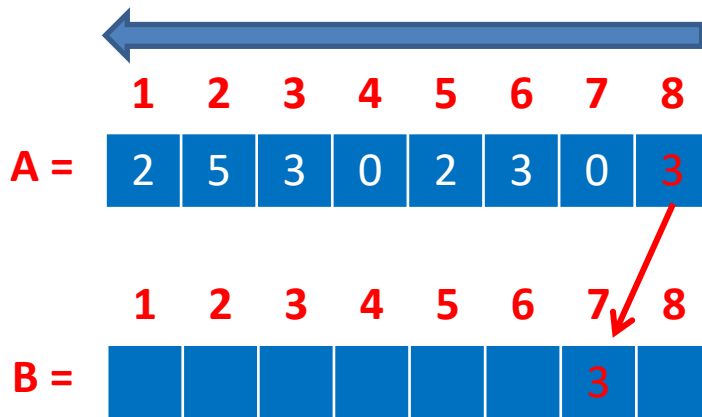
Сортування підрахунком

Крок 3: “Злиття” елементів з масиву A в масив B у відсортованому порядку



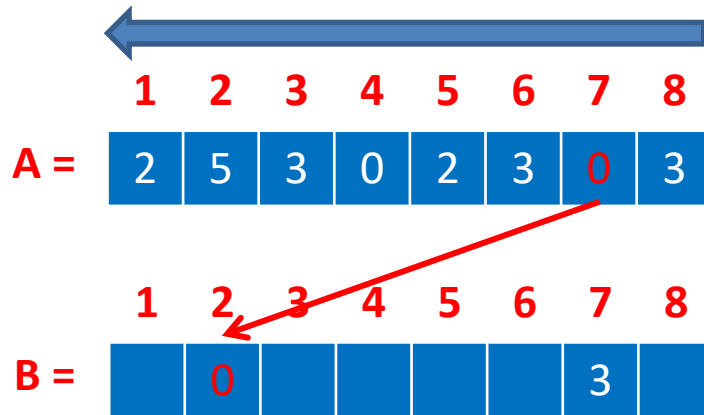
Рухаємось з кінця масиву до початку !!!
for (j = n; j > 0; j--)

1) $j = 8 \rightarrow A[8] = 3 \rightarrow C[A[8]] = C[3] = 7 \longrightarrow B[7] = 3; C[3] = C[3] - 1 = 6$



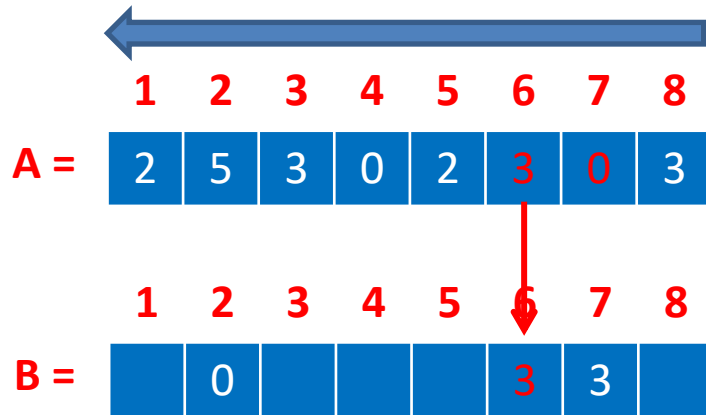
Сортування підрахунком

2) $j = 7 \rightarrow A[7] = 0 \rightarrow C[A[7]] = C[0] = 2 \Rightarrow B[2] = 0; C[0] = C[0] - 1 = 1$



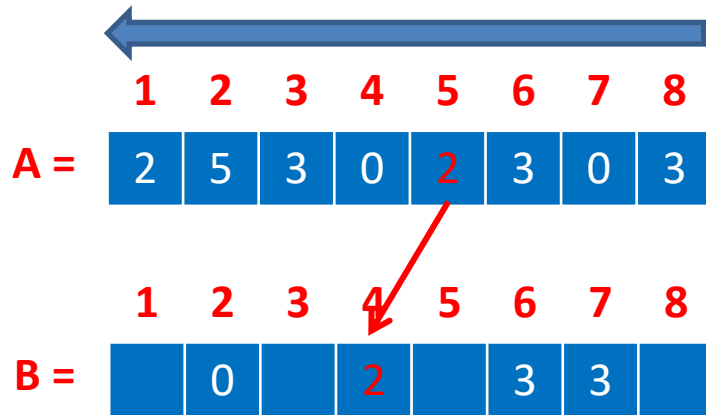
Сортування підрахунком

3) $j = 6 \rightarrow A[6] = 3 \rightarrow C[A[6]] = C[3] = 6 \longrightarrow B[6] = 3; C[3] = C[3] - 1 = 5$



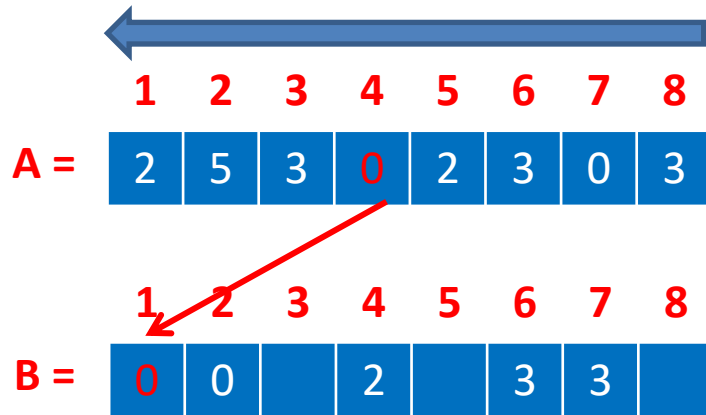
Сортування підрахунком

4) $j = 5 \rightarrow A[5] = 2 \rightarrow C[A[5]] = C[2] = 4 \longrightarrow B[4] = 2; C[2] = C[2] - 1 = 3$



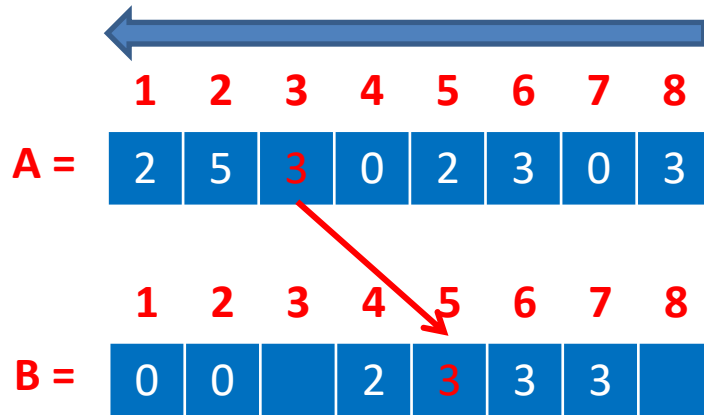
Сортування підрахунком

5) $j = 4 \rightarrow A[4] = 0 \rightarrow C[A[4]] = C[0] = 1 \implies B[1] = 0; C[0] = C[0] - 1 = 0$



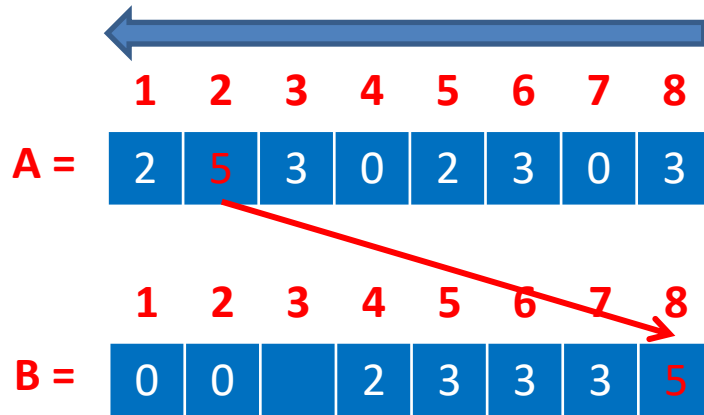
Сортування підрахунком

6) $j = 3 \rightarrow A[3] = 3 \rightarrow C[A[3]] = C[3] = 5 \rightarrow B[5] = 3; C[3] = C[3] - 1 = 4$



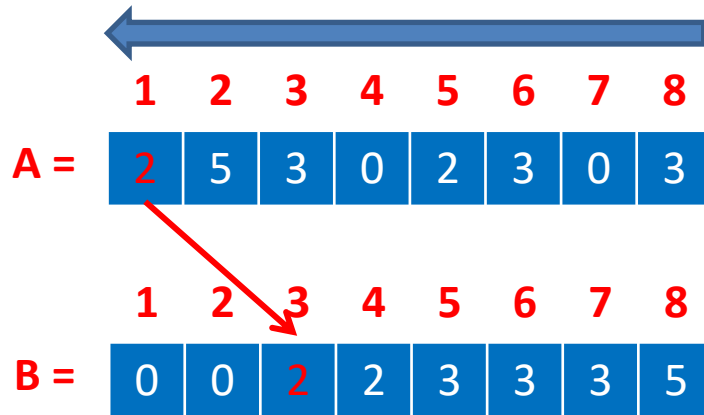
Сортування підрахунком

7) $j = 2 \rightarrow A[2] = 5 \rightarrow C[A[2]] = C[5] = 8 \longrightarrow B[8] = 5; C[5] = C[5] - 1 = 7$



Сортування підрахунком

8) $j = 1 \rightarrow A[1] = 2 \rightarrow C[A[1]] = C[2] = 3 \longrightarrow B[3] = 2; C[2] = C[2] - 1 = 2$



Сортування підрахунком

	1	2	3	4	5	6	7	8
B =	0	0	2	2	3	3	3	5

$i =$	0	1	2	3	4	5
C =	0	2	2	4	7	7

	1	2	3	4	5	6	7	8
B =	0	0	2	2	3	3	3	5



	1	2	3	4	5	6	7	8
A =	0	0	2	2	3	3	3	5

Delete B

	1	2	3	4	5	6	7	8
B =	0	0	2	2	3	3	3	5

Delete C

$i =$	0	1	2	3	4	5
C =	0	2	2	4	7	7

Сортування підрахунком

CountingSort(A, B, k)

Динамічно створити масив C[0...k]

```
1 for i = 0 to k      }  $\Theta(k)$ 
2     do C[i] = 0
3 for j = 1 to length[A] }  $\Theta(n)$ 
4     do C[A[j]] = C[A[j]] + 1
// в C[i] зберігається кількість елементів, які дорівнюють i
5 for i = 1 to k      }  $\Theta(k)$ 
6     do C[i] = C[i] + C[i - 1]
// в C[i] зберігається кількість елементів, які не більші за i
7 for j = length[A] downto 1 // цикл за спаданням }  $\Theta(n)$ 
8     do B[C[A[j]]] = A[j]
9     C[A[j]] = C[A[j]] - 1
```

Часова складність сортування підрахунком $T(n) = \Theta(2n + 2k) = \Theta(n + k)$ – лінійна.

Але чи все так добре?

Сортування підрахунком

Приклад 1. **Поганий приклад**

$n = 100 = 10^2$ – кількість елементів

$k = 1000000 = 10^6$ – максимальне значення елементів



$$T(n) = \Theta(10^6 + 10^2) = \underline{\Theta(10^6)}$$



Якщо використати сортування “бульбашкою”, то

$$T(n) = \Theta(n^2) = \underline{\Theta(10^4)}$$

Приклад 2. **Хороший приклад**

$n = 1000000 = 10^6$ – кількість елементів

$k = 100 = 10^2$ – максимальне значення елементів



$$T(n) = \Theta(10^6 + 10^2) = \underline{\Theta(10^6)}$$



Якщо використати сортування “бульбашкою”, то

$$T(n) = \Theta(n^2) = \underline{\Theta(10^{12})}$$

Якщо використати швидке сортування, то

$$T(n) = \Theta(n \log_{10} n) = \underline{\Theta(10^6 \times 6)}$$

**Сортування
підрахунком добре
працює, коли $k = O(n)$**



$$T(n) = \Theta(n + k) = \Theta(n)$$

Стійке сортування

Чому останній цикл в сортуванні злиттям йде за-спаданням?

Що буде якщо зробити цикл за-зростанням?

Означення:

Алгоритм сортування є **стійким**, якщо елементи з одним й тим самим значенням знаходяться у вихідному масиві в **тому самому порядку**, що й у вхідному.



Якщо розглядати сортування підрахунком як самодостатню (окрему) задачу, то стійкість чи нестійкість алгоритму не впливає на результат.

Однак, при використанні сортування підрахунком як частини іншої задачі, стійкість алгоритму має вирішальне значення.

Сортування за розрядами

Постановка задачі:

Вхід: послідовність n чисел $[a_1, a_2, \dots, a_n]$, де:

a_i – число, яке має d – розрядів.

Вихід: перестановка $[a'_1, a'_2, \dots, a'_n]$ вхідної послідовності таким чином, що для всіх її членів виконується співвідношення $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Приклад:

$A = [329, 457, 657, 839, 436, 720, 355]$  **$n = 7, d = 3$**

Сортування підрахунками тут не підходить (**поганий приклад**), оскільки **$n = 7, k = 839 \rightarrow n = O(k)$**

(або **$k = 999$** , якщо нам не відомий максимальний елемент, а тільки кількість розрядів **$d = 3$**)

Що буде, якщо спробувати посортувати дані за кожним розрядом окремо?

Тоді виникає питання, в якому напрямку сортувати:

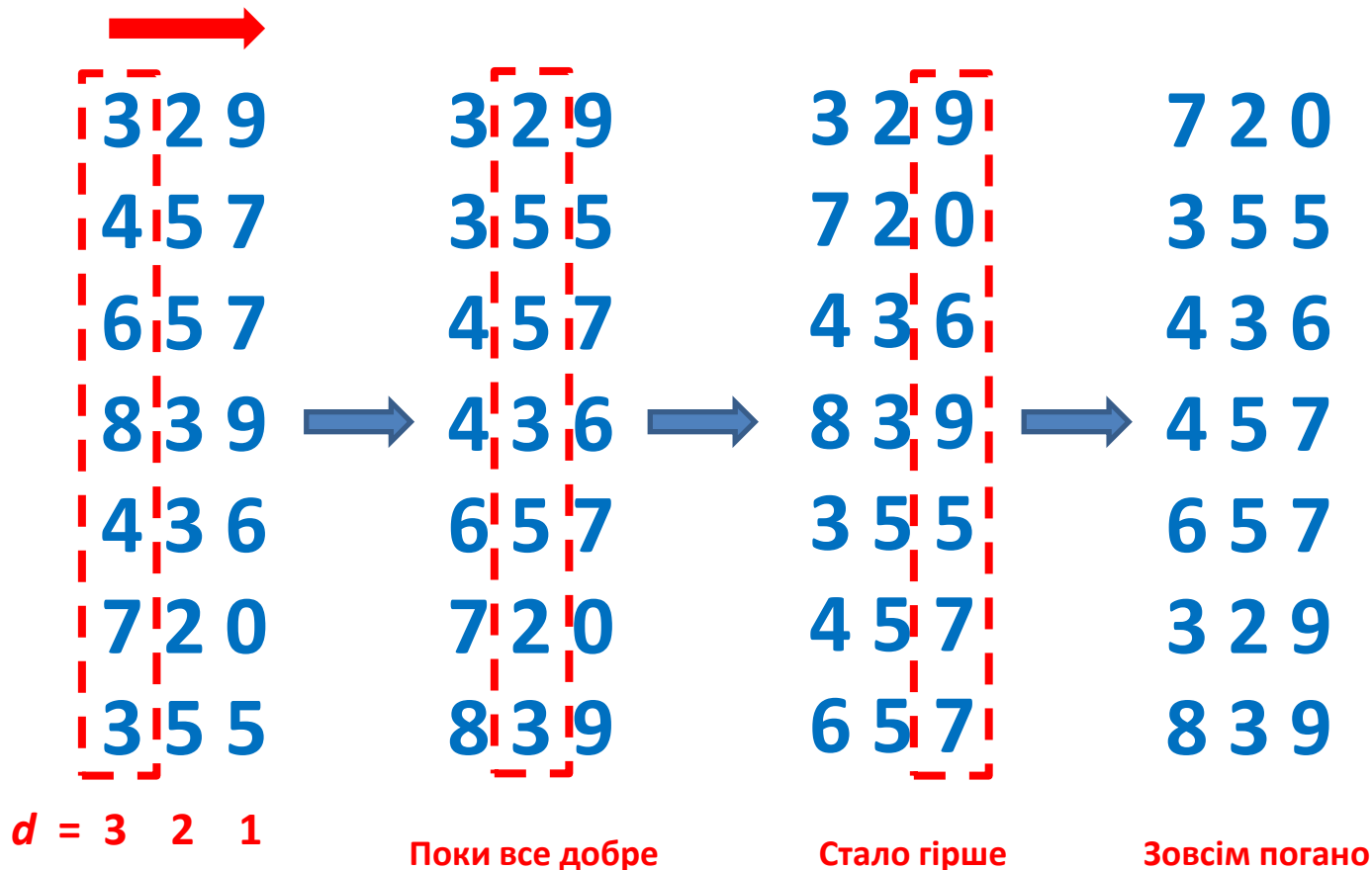
- від старшого розряду до молодшого (зліва-направо)**
- чи**
- від молодшого розряду до старшого (справа-наліво)**

?


 $d = 3 \ 2 \ 1$
329

Сортування за розрядами

Спробуємо від старшого розряду до молодшого:

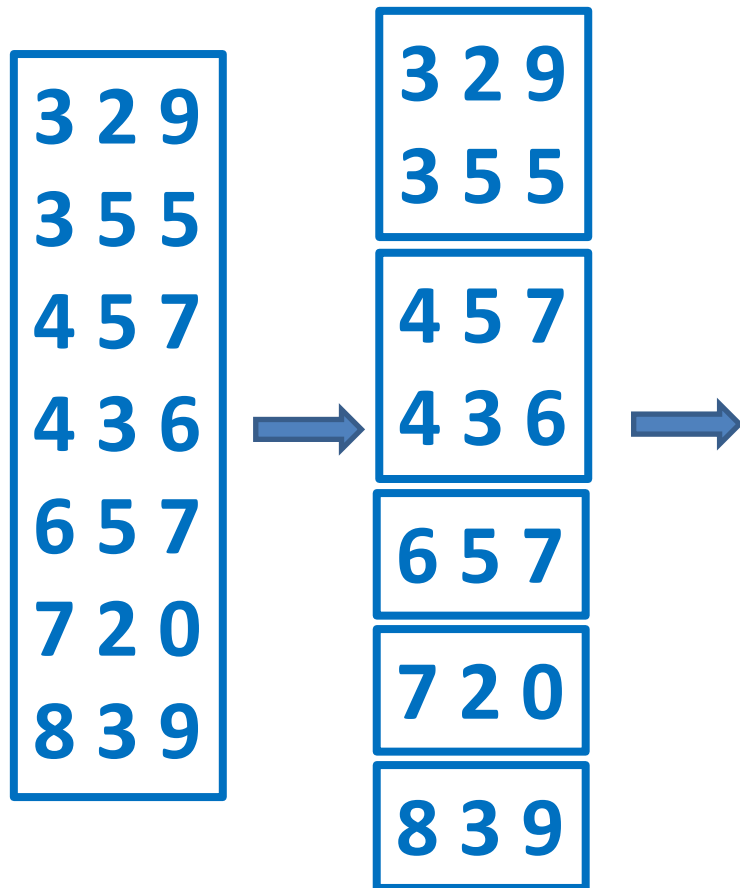


Це приклад нестійкого сортування:

при сортуванні молодших розрядів втрачається сортування в старших

Сортування за розрядами

Одним з виходів з цієї ситуації це сортувати не весь масив зразу, а розбити його на підмасиви з однаковим старшим розрядом і здійснювати сортування в підмасивах:



Тут нам потрібно чотири масиви, під кожний старший розряд: 3, 4, 6, 7, 8



А в загальному таких масивів 10:
0...9



Для кожного з цих 10 масивів на потрібно ще 10 під наступний розряд і т.д.



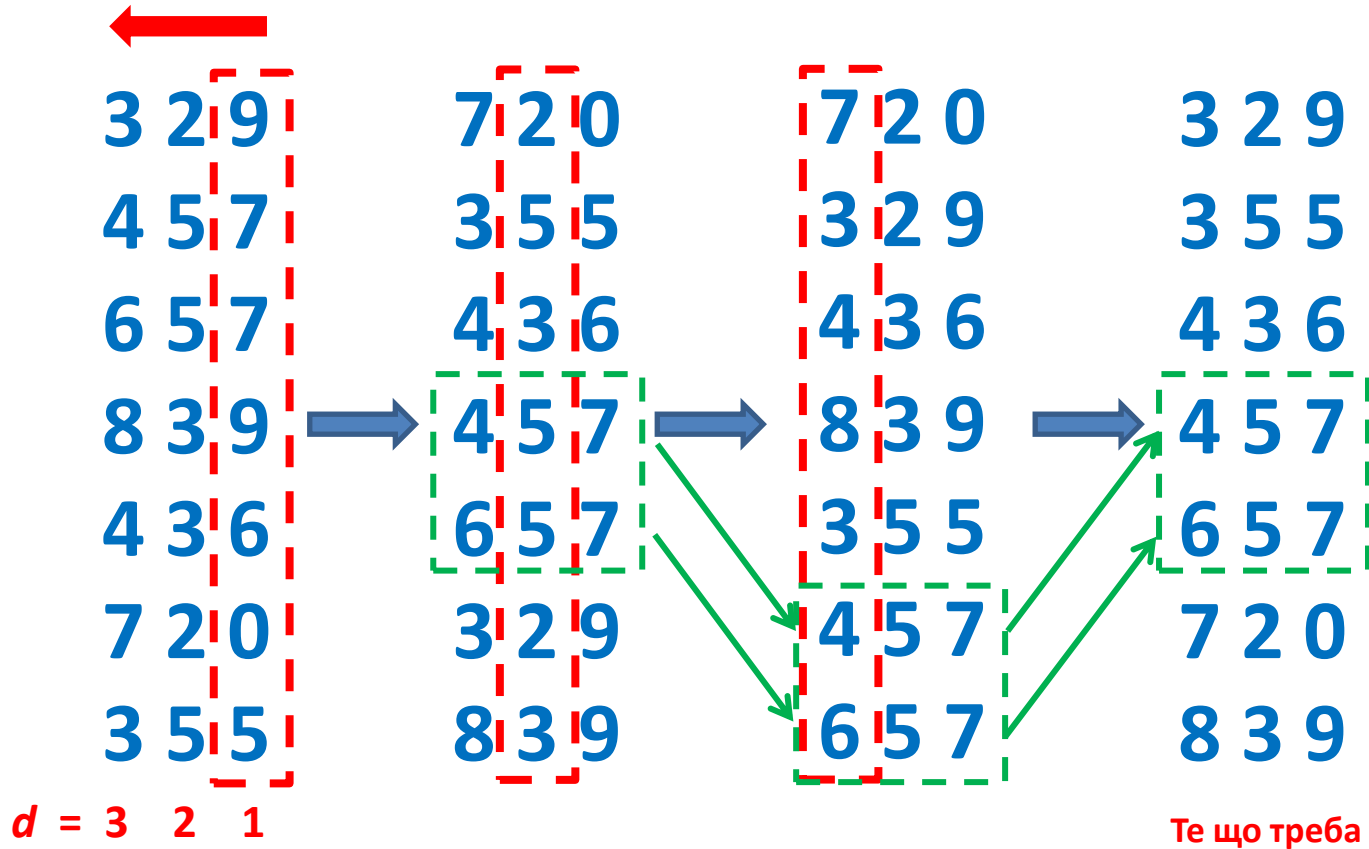
Тобто кількість допоміжних масивів зростає в геометричній прогресії з кроком 10



АБСОЛЮТНО НЕЕФЕКТИВНО

Сортування за розрядами

Але спробуємо сортувати від молодшого розряду до старшого:



Це приклад стійкого сортування:

елементи з однаковими молодшими розрядами зберігають свій початковий порядок

Зауваження:

Аналогічним чином потрібно сортувати
і іншу інформацію, наприклад дати:

Рік Місяць День

Сортування за розрядами

RadixSort(A, d)

1 **for** i = 1 **to** d // Стійке сортування

2 **do** Стійке сортування масиву A за i-м розрядом

Наприклад : стійке сортування підрахунком

Як з d розрядного числа виділити i -ий розряд ? $1 \leq i \leq d$

$d = 4 \quad 3 \quad 2 \quad 1$

$7 \boxed{6} 5 4 \xrightarrow{\quad} 7654 / 100 = 76 \xrightarrow{\quad} 76 \bmod 10 = 6$

Цілочисельне ділення

$$\text{Число} / 10^{i-1} \bmod 10; 1 \leq i \leq d$$

Сортування за розрядами

Теорема.

Нехай є n d -значних чисел, в яких кожна цифра приймає одне з k можливих значень. Тоді алгоритм RadixSort дозволяє виконати коректне сортування цих чисел за час $\Theta(d(n + k))$, якщо стійке сортування, яке використовуються цим алгоритмом, має час роботи $\Theta(n + k)$.

Доведення.

Коректність сортування за розрядами доводиться методом математичної індукції по стовпцях, за якими відбувається сортування. Аналіз часу роботи алгоритму залежить від того, який з методів стійкого сортування використовується в якості допоміжного алгоритму. Якщо кожна цифра належить інтервалу від 0 до $k - 1$, і k не надто велике, то сортування за розрядами – оптимальний вибір. Для обробки кожної з d цифр n чисел потрібний час $\Theta(n + k)$, а всі цифри будуть опрацьовані алгоритмом сортування за розрядами за час $\Theta(d(n + k))$. ■

Якщо відомо, що $k = O(n)$ і d – стала, то час роботи сортування за розрядами **лінійно залежить від кількості вхідних елементів**.

Сортування за розрядами

Переваги сортування за розрядами:

Лінійність $\rightarrow T(n) = O(n)$, якщо $k = O(n)$ і d – стале.

Недоліки сортування за розрядами:

- Обмеження на вхідні дані.
- Використання додаткової пам'яті.
- Приховані сталі множники.

ДЯКУЮ ЗА УВАГУ!